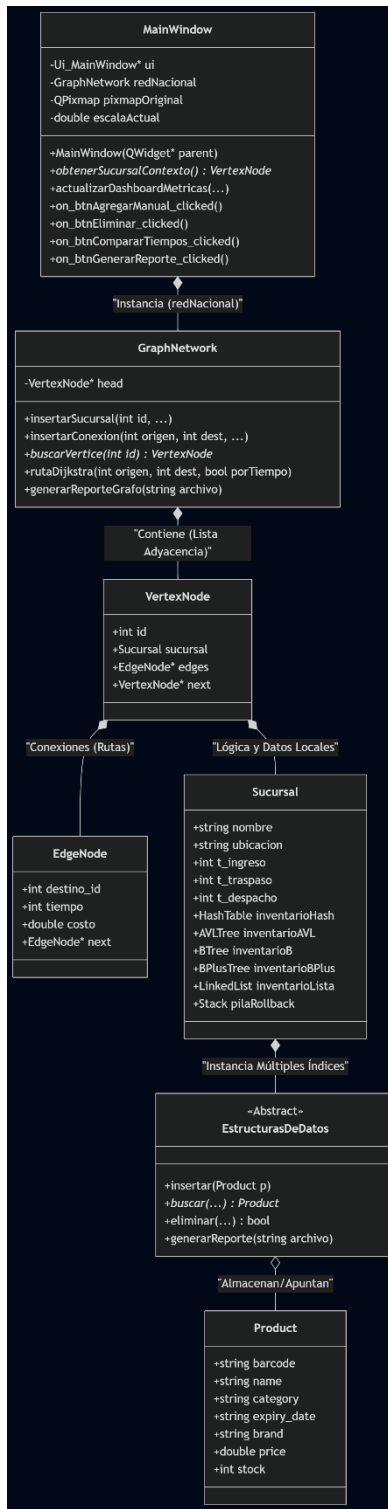


# SISTEMA DE GESTION DE CATÁLOGO DE PRODUCTOS DE SUPERMERCADO

## 1. DIAGRAMA DE CLASES:



## 2. TAD's (Especificación de tipos abstractos de datos)

En esta sección definimos el comportamiento formal de las estructuras implementadas. Un único objeto entidad (Product) reside en la memoria dinámica y es referenciado simultáneamente por múltiples estructuras indexadas. A continuación, se detalla la especificación de cada Tipo Abstracto de Dato (TAD) implementado.

### 1. Grafo Ponderado Bidireccional (Red Logística)

**Propósito:** Modelar la topología física nacional.

**Implementación Interna:** Lista de Adyacencia (Arreglo dinámico o lista enlazada de vértices, donde cada vértice posee una lista enlazada de aristas).

- **Vértices (Nodos):** Representan geográficamente a cada Sucursal. Almacenan un identificador único (ID), nombre, ubicación y las colas de procesamiento interno (Ingreso, Traspaso, Despacho).
- **Aristas (Conexiones):** Representan las rutas (carreteras) viables entre dos sucursales. Son **bidireccionales** por defecto y poseen un **peso multidimensional**:
  - $W_1$ : Tiempo de tránsito (en segundos).
  - $W_2$ : Costo económico del flete (en Quetzales).
- **Operación Crítica: Ruta Más Corta (Dijkstra):** Algoritmo de exploración de caminos que, dada una sucursal de origen y una de destino, calcula la ruta óptima minimizando la sumatoria total del peso elegido por el usuario (minimizar tiempo  $W_1$  o minimizar costo  $W_2$ ). Su complejidad es  $O(E \log V)$  utilizando colas de prioridad, o  $O(V^2)$  en implementaciones estándar.

### 2. Tabla Hash (Índice Primario)

**Propósito:** Proveer acceso instantáneo y determinista a cualquier producto mediante su llave primaria única (Código de Barras).

**Implementación Interna:** Arreglo dinámico con resolución de colisiones por **Encadenamiento Separado** (*Separate Chaining*).

- **Función de Dispersión  $h(k)$ :** Algoritmo matemático (ej. *MurmurHash* o suma polinomial de caracteres) que convierte el *string* del código de barras en un índice numérico dentro de los límites del arreglo.

- **Manejo de Colisiones:** Cada "bucket" (ranura) del arreglo es la cabeza de una lista enlazada simple. Si dos códigos de barras producen el mismo índice, el nuevo producto se inserta en la lista correspondiente.
- **Rendimiento:**
  - Búsqueda/Inserción/Eliminación (Promedio):  $O(1)$ .
  - Búsqueda (Peor Caso, colisión masiva):  $O(n)$ .
- **Justificación de Diseño:** Se eligió el encadenamiento sobre el direccionamiento abierto para evitar la *Aglomeración Primaria* (Primary Clustering) y el costoso uso de "lápidas" (*Tombstones*) al eliminar productos, garantizando eliminaciones atómicas en  $O(1)$ .

### 3. Árbol AVL (Índice Alfabético)

**Propósito:** Mantener el catálogo de inventario estrictamente ordenado por nombre y garantizar búsquedas predecibles.

**Implementación Interna:** Árbol Binario de Búsqueda Auto-Balanceable.

- **Regla de Balanceo:** Para cada nodo, la diferencia de altura entre su subárbol izquierdo y derecho (Factor de Equilibrio) nunca debe ser mayor a 1 ni menor a -1 ( $-1 \leq FE \leq 1$ ).
- **Mantenimiento:** Tras cada inserción o eliminación, el árbol recalcula las alturas. Si detecta un desequilibrio, ejecuta **Rotaciones** (Simples a la Izquierda/Derecha o Dobles Izquierda-Derecha/Derecha-Izquierda) para restaurar el factor de equilibrio.
- **Rendimiento:**
  - Búsqueda/Inserción/Eliminación:  $O(\log n)$  garantizado en todos los casos.
- **Operación Crítica:** Recorrido **In-Order** (Izquierda-Raíz-Derecha) para exportar el inventario completo ordenado alfabéticamente de la 'A' a la 'Z'.

### 4. Árbol B (Índice Temporal)

**Propósito:** Agrupar productos para optimizar las consultas por rangos específicos de fechas de expiración.

**Implementación Interna:** Árbol de búsqueda multidimensional con grado mínimo  $t=2$ .

- **Estructura del Nodo:** Un nodo puede contener hasta  $2t + 1$  claves (fechas de expiración) y poseer hasta  $2t + 2$  hijos.
- **Balanceo Estructural:** El árbol crece y decrece desde la raíz (crecimiento horizontal). Todos los nodos hoja se mantienen estrictamente al mismo nivel de profundidad. Si un nodo se llena, sufre una operación de división (*Split*) empujando la clave mediana hacia el nodo padre.
- **Rendimiento:**
  - Búsqueda/Inserción/Eliminación:  $O(\log_t n)$ .
- **Justificación:** Al ser "ancho y poco profundo", el Árbol B reduce drásticamente el número de saltos de memoria requeridos para encontrar un grupo de fechas, en comparación con un árbol binario tradicional.

## 5. Árbol B+ (Índice Categórico)

**Propósito:** Permitir la extracción masiva y secuencial de todos los productos que pertenezcan a una misma categoría semántica (ej. "Lácteos").

**Implementación Interna:** Variante del Árbol B (grado  $t=2$ ) optimizado para consultas de bases de datos.

- **Nodos Internos:** Solo almacenan "Claves Guía" (nombres de categorías) para dirigir la búsqueda. NO almacenan punteros a los productos reales.
- **Nodos Hoja (Hojas de Datos):** Es el único nivel que contiene los punteros a los objetos Product reales.
- **Enlace Secuencial:** Lo más crítico del diseño es que los nodos hoja están conectados entre sí formando una **Lista Enlazada**.
- **Operación Crítica:** Búsqueda por Categoría. El sistema desciende logarítmicamente  $O(\log_t n)$  hasta encontrar la primera hoja que contenga la categoría "Lácteos". A partir de ahí, detiene la búsqueda arbórea y simplemente itera linealmente hacia la derecha  $O(k)$  extrayendo todos los productos hasta que la categoría cambie, evitando recorrer el árbol repetidamente.

## 6. Estructuras Lineales de Control (Pilas y Colas)

**Implementación Interna:** Listas Enlazadas Simples o Arreglos Circulares estáticos.

- **Pila (Stack) de Rollback:**

- **Propósito:** Almacenar el historial de inserciones para permitir la función "Deshacer".
- **Comportamiento:** Regla **LIFO** (Last-In, First-Out). El último producto insertado en la sucursal es el primero que se elimina al solicitar un Rollback. Operaciones Push (Apilar) y Pop (Desapilar) en  **$O(1)$** .
- **Colas (Queues) de Simulación:**
  - **Propósito:** Modelar las demoras físicas (cuellos de botella) dentro de una sucursal durante un despacho logístico.
  - **Comportamiento:** Regla **FIFO** (First-In, First-Out). Cada sucursal posee 3 colas distintas (Ingreso, Preparación, Salida). Los paquetes entran por el Rear (final) y son procesados por el Front (frente). Operaciones Enqueue (Encolar) y Dequeue (Desencolar) en  $O(1)$ .